

Informe Taller 4: Planificación Automatizada

Inteligencia Artificial – ISIS-1611

Nicolás Rey
n.reyl@uniandes.edu.co
202321258

Esteban Preciado
e.preciado@uniandes.edu.co
202411233

Tomás Torres
t.torresm@uniandes.edu.co
202313980

18 de mayo de 2026

Notación

A lo largo del informe se usará la siguiente notación:

- n : número de flujos del dominio (tamaño del estado).
 - a ó A : número de acciones instanciadas (groundings).
 - b : factor de ramificación efectivo (acciones aplicables promedio por estado).
 - d : profundidad del plan óptimo.
 - f : tiempo de cómputo de la heurística por nodo.
 - F : cantidad total de flujos posibles.
 - N : cantidad de nodos en la frontera.
-

Punto 6a: Análisis de los algoritmos

Para cada uno de los algoritmos implementados en los puntos 1 a 5, se reporta a continuación su complejidad espacial, complejidad temporal, completitud, optimalidad y, cuando corresponde, las propiedades de las heurísticas asociadas. Al final se incluyen tablas comparativas con métricas empíricas.

Análisis del Punto 1 – Modelado del dominio y verificación de aplicabilidad

El Punto 1 no corresponde a un algoritmo de búsqueda, sino a la infraestructura sobre la que corren todos los planificadores posteriores. Aun así, sus funciones (`is_applicable`, `apply_action` y `get_applicable_actions`) se invocan miles de veces dentro de cada búsqueda, por lo que su costo es determinante.

Complejidad espacial $O(n + a)$: El estado se representa como un `frozenset` de flujos, ocupando $O(n)$ memoria por estado. La función `get_applicable_actions` construye una lista con todos los groundings aplicables, lo que requiere $O(a)$ memoria adicional. `apply_action` construye un nuevo estado de tamaño $O(n)$ mediante operaciones de conjuntos.

Complejidad temporal $O(a \cdot p)$: La verificación `is_applicable` es $O(p)$ donde p es el número de flujos en la precondition de la acción (las operaciones de pertenencia sobre `frozenset` son

$O(1)$ amortizado). `get_applicable_actions` enumera los a groundings y verifica precondiciones, dando $O(a \cdot p)$. En el dominio de rescate, la acción `Move(r, from, to)` domina porque genera $O(C^2)$ groundings donde C es el número de celdas libres, aunque la mayoría son podadas por el predicado `Adjacent`.

Complejidad: Las funciones son determinísticas y completas por construcción: siempre devuelven el resultado correcto en tiempo finito. No son algoritmos de búsqueda, por lo que el concepto de completitud no aplica en el sentido tradicional.

Optimalidad: No aplica. Estas funciones son operaciones simbólicas sobre estados, no producen planes.

Observación práctica: `get_applicable_actions` es el cuello de botella oculto de todos los planificadores. En layouts grandes el número de `Move` instanciados crece rápidamente y cada expansión de estado paga ese costo. Por esta razón, la eficiencia del modelo del dominio impacta directamente la viabilidad práctica de todos los algoritmos siguientes.

Análisis del Punto 2 – Búsqueda hacia adelante (Forward BFS)

El planificador hacia adelante parte del estado inicial y aplica acciones hasta encontrar un estado que satisfaga el objetivo, explorando el espacio de estados en anchura.

Complejidad espacial $O(b^d \cdot n)$: La frontera (cola FIFO) y el conjunto `visited` crecen exponencialmente con la profundidad del plan. Cada estado almacenado ocupa $O(n)$ memoria, y en el peor caso se almacenan $O(b^d)$ estados antes de encontrar el objetivo.

Complejidad temporal $O(b^d \cdot a \cdot p)$: BFS expande en el peor caso todos los estados a profundidad $\leq d$, lo que da $O(b^d)$ expansiones. Cada expansión paga el costo de `get_applicable_actions` ($O(a \cdot p)$) más el costo de verificar si los nuevos estados están en `visited` ($O(1)$ amortizado por estado).

Complejidad: El algoritmo es completo en espacios de estados finitos. El uso del conjunto `visited` evita revisitar estados y garantiza terminación. Como el espacio de fuentes es finito y los layouts tienen un número limitado de configuraciones alcanzables, BFS siempre encontrará un plan si existe.

Optimalidad: El algoritmo es óptimo cuando todas las acciones tienen costo uniforme, que es el caso del dominio de rescate (cada `Action.cost = 1`). BFS minimiza el número de pasos del plan, lo que coincide con minimizar el costo bajo costos uniformes. Si los costos fueran variables, BFS perdería la garantía de optimalidad y habría que usar búsqueda de costo uniforme o A^* .

Observación práctica: Aunque hay muchos groundings de `Move`, en un estado dado solo son aplicables los movimientos desde la celda actual del robot y, a lo sumo, una manipulación (`PickUp`, `PutDown`, `Rescue` o `SetupSupplies`). Por eso $b \approx 2-4$ típicamente, pero d crece linealmente con la distancia Manhattan más el número de manipulaciones requeridas, lo que vuelve b^d inmanejable para mapas grandes.

Análisis del Punto 3 – Backward search

Complejidad espacial $O(N \cdot (f + d))$: La complejidad espacial depende de la cantidad de nodos desplegados por la respuesta, definido como N , puesto que corresponde a la frontera del problema, la cual backward search guarda en su totalidad, de la profundidad del plan, representada por d , y de f , el cual es el valor de la heurística. El algoritmo debe almacenar planes parciales, creciendo la cantidad de estados y refinamientos según el número de pacientes.

Complejidad temporal $O(f \cdot b^d)$: La complejidad temporal depende del factor de ramificación, representado por b , la profundidad del plan, representada por d , y del tiempo en calcular la heurística por cada nodo, calificado como f . En el peor de los casos aún se tienen que explorar todos los nodos, por lo que el cálculo de la heurística solo multiplica el tiempo esperado.

Complejitud: El algoritmo sí se puede considerar completo, puesto que la cola de prioridad eventualmente desplegará todos los nodos y la lista de visitados previene que se formen ciclos infinitos.

Optimalidad: El algoritmo no garantiza encontrar el plan de menor costo, no pudiendo considerarse óptimo, debido a que su *greedy search* no garantiza que el costo acumulado sea mayor, solo centrándose en las ganancias “a corto plazo”.

¿En qué tipos de problemas la búsqueda regresiva es más eficiente? La búsqueda regresiva resulta más eficiente que la búsqueda hacia adelante cuando el *objetivo es muy específico* (pocos fluentes describen la meta) frente a un *estado inicial muy rico* (muchísimos fluentes verdaderos), porque arrancar desde el objetivo reduce drásticamente el factor de ramificación: solo se consideran acciones cuyo `add_list` interseca el objetivo actual, en lugar de todas las acciones aplicables al estado. En el dominio de rescate esto es claro: el objetivo final es solo `Rescued(patient_0)`, mientras que el estado inicial contiene decenas de fluentes (`At`, `Free`, `Adjacent`, `MedicalPost`, etc.). La búsqueda regresiva también gana en dominios donde la mayoría de las acciones son irrelevantes para el objetivo (muchas acciones nunca tocan los fluentes del goal), pues estas se podan automáticamente. Por el contrario, pierde frente a la búsqueda hacia adelante cuando el objetivo está formado por muchos fluentes o cuando los subobjetivos contienen fluentes estáticos falsos que no se podan correctamente.

Análisis del Punto 4a – Heurística de ignorar precondiciones

Complejidad espacial $O(A + F)$: La complejidad de la heurística depende de dos factores, que son el número de acciones posibles (A) y los fluentes posibles (F). A solo se calcula una vez al inicio del código, y F en el peor de los casos son todos los fluentes posibles.

Complejidad temporal $O(A \cdot f^2)$: La complejidad depende principalmente de la cantidad de veces sobre las que se itera sobre todas las acciones (A) y la cantidad de veces que se tiene que repetir el ciclo. Cada que se repite se está atacando al menos a un fuente, por lo que la cantidad de veces máxima que se itera es la cantidad de fluentes que hay (f). Y debido a que también se calcula la intersección sobre los fluentes dentro del loop, en el peor de los casos se recorre f veces dos veces (f^2).

Complejitud: El algoritmo A^* es completo porque eventualmente va a encontrar un plan que solucione el problema, puesto que el mapa es de tamaño fijo, y porque todas las acciones tienen el mismo costo, el cual siempre es positivo, lo que garantiza que el algoritmo pueda explorar los estados de forma sistémica hasta encontrar una solución.

Optimalidad: El algoritmo A^* sí garantiza encontrar el plan de menor costo dentro de las opciones posibles, siempre y cuando se use una heurística admisible, es decir, que no sobreestime el costo para lograr dicho objetivo. La heurística `ignorePreconditionsHeuristic` cumple, ya que trabaja sobre una versión simplificada donde no se pierde progreso al ejecutar acciones. Además, como las acciones tienen costo uniforme y positivo, A^* puede asegurar una solución óptima si existe un plan válido.

¿La heurística es consistente? La heurística `ignorePreconditionsHeuristic` es consistente dentro del dominio del problema, debido a que siempre que se calcule, tendrá en cuenta todos los nodos sin importar sus precondiciones. Esto implica que siempre se irá por el nodo que lo

acerque más a la meta, por lo que el siguiente nodo no puede tener un camino mejor, asegurando que $h(n) \leq c(n, n') + h(n')$.

¿La heurística es admisible? La heurística es admisible, puesto que toda heurística consistente es admisible.

Ejemplo concreto de admisibilidad (layout tinyBase): En el estado inicial de `tinyBase` el objetivo es `{Rescued(patient_0)}` (un único fuente faltante). La heurística `ignorePreconditionsHeuristic` devuelve $h = 1$, porque una sola aplicación de la acción `Rescue` añadiría el fuente faltante a la cobertura. Sin embargo, el plan óptimo real requiere 9 acciones primitivas (mover hasta los suministros, recogerlos, llevarlos al puesto, activarlos, ir por el paciente, recogerlo, llevarlo al puesto, dejarlo y rescatarlo). Se cumple $h(s_0) = 1 \leq 9 = h^*(s_0)$, por lo que la heurística *no* sobreestima el costo real: es admisible.

Análisis del Punto 4b – Heurística de ignorar listas de borrado

Complejidad espacial $O(n + a)$: La complejidad de la heurística depende de dos factores, que son el número de fuentes del estado, representado por n , y el número de acciones disponibles, representado por a , por lo que almacena el estado y el conjunto de acciones disponibles desde ese estado.

Complejidad temporal $O(b^d)$: La complejidad que utiliza el planificador A^* depende de dos factores principales, que son el número de acciones aplicables desde un estado, representado por b , y de la profundidad de la solución, que en el peor caso explora una cantidad exponencial de estados antes de alcanzar el objetivo deseado, representada por d .

Complejidad: El algoritmo A^* es completo porque eventualmente va a encontrar un plan que solucione el problema, puesto que el mapa es de tamaño fijo, y porque todas las acciones tienen el mismo costo, el cual siempre es positivo, lo que garantiza que el algoritmo pueda explorar los estados de forma sistemática hasta encontrar una solución.

Optimalidad: El algoritmo A^* sí garantiza encontrar el plan de menor costo dentro de las opciones posibles, siempre y cuando se use una heurística admisible. La heurística `ignoreDeleteListsHeuristic` cumple, ya que trabaja sobre una versión simplificada donde no se pierde progreso al ejecutar acciones. Además, como las acciones tienen costo uniforme y positivo, A^* puede asegurar una solución óptima si existe un plan válido.

¿La heurística es admisible? La heurística `ignoreDeleteListsHeuristic` sí es admisible, puesto que no sobreestima el costo real de la solución. Como la heurística trabaja sobre una versión relajada del problema, en la que no se eliminan fuentes del estado, lo que facilita encontrar una solución, sumado a que todas las acciones tienen el mismo costo, dando lugar a que el costo estimado por la heurística siempre sea menor o igual al costo real.

¿La heurística es consistente? La heurística `ignoreDeleteListsHeuristic` es consistente dentro del dominio del problema, porque al aplicar una acción en el problema relajado no se eliminan fuentes, sino que el estado solo mantiene o aumenta el progreso. El costo estimado cambia de manera gradual al ejecutar acciones, sin reducirse más de lo que cuesta avanzar realmente.

¿Cuál de las dos heurísticas es más informativa? Empíricamente, en el dominio de rescate ambas heurísticas terminan dando estimaciones muy similares para los layouts simples probados, expandiendo el mismo número de nodos en `tinyBase`, `cornerRescue` y `smallRescue` (ver Tabla 1). Esto se debe a que el objetivo (`Rescued(patient_0)`) es un único fuente que ninguna acción del dominio “deshace” una vez que se logra, por lo que la relajación de listas de borrado no aporta información adicional frente a la relajación de precondiciones para este goal

específico. En términos teóricos, `ignoreDeleteListsHeuristic` es *potencialmente* más informativa porque considera la dinámica del estado (no solo qué acciones añaden el goal, sino qué secuencia de acciones lo construye), pero también es *computacionalmente más cara* por iteración, ya que su hill-climbing llama a `get_applicable_actions` en cada paso. En la práctica, `ignorePreconditionsHeuristic` ofrece una mejor relación costo/beneficio para este dominio.

Análisis del Punto 5a – Definición de HLAs y refinamientos

La jerarquía implementada para el dominio de rescate consta de cuatro HLAs:

- `Navigate(from, to)`: se refina en una secuencia de acciones primitivas `Move` que recorre el camino más corto entre ambas celdas (BFS sobre la grilla).
- `PrepareSupplies(s, m)`: se refina como `[Navigate(robot, s_loc), Pickup(s), Navigate(s_loc, m), SetupSupplies(s, m)]`.
- `ExtractPatient(p, m)`: se refina como `[Navigate(robot, p_loc), Pickup(p), Navigate(p_loc, m), PutDown(p), Rescue(p)]`.
- `FullRescueMission(s, p, m)`: composición de HLAs: `[PrepareSupplies(s, m), ExtractPatient(p, m)]`.

Complejidad espacial $O(b^d)$: La complejidad espacial depende del número de refinamientos posibles por cada tarea, representado por b , como de la profundidad del plan jerárquico, representada por d . El algoritmo almacena planes parciales en la frontera, esto mientras expande los refinamientos hasta obtener un plan completamente primitivo.

Complejidad temporal $O(b^d)$: La complejidad temporal depende del factor de ramificación, representado por b , y de la profundidad jerárquica del plan, representada por d . En el peor caso, el algoritmo puede explorar una cantidad exponencial de refinamientos y posibles órdenes de rescate antes de encontrar un plan que satisfaga la solución.

Complejidad: El algoritmo HTN sí es completo, esto mientras exista un conjunto finito de refinamientos, y que dichos refinamientos permitan eventualmente llegar a acciones primitivas a partir de las HLAs. Las HLAs tienen refinamientos definidos en un espacio de búsqueda finito, por lo que el algoritmo puede encontrar una solución válida en caso de que exista.

Optimalidad: El planificador HTN no garantiza encontrar el plan de menor costo, debido a que el algoritmo explora refinamientos jerárquicos mediante BFS, y el resultado depende del orden en el que los refinamientos fueron definidos. Por todo esto, el algoritmo sí encuentra un plan válido, pero no hay garantía de que este sea el más óptimo.

¿La heurística es admisible? La heurística no se puede considerar admisible, puesto que el planificador HTN no utiliza una heurística que dé una estimación, como sí hacía el 4b. La planificación en HTN se basa en refinamientos jerárquicos, derivados de acciones de alto nivel.

¿La heurística es consistente? La heurística no se puede considerar consistente, esto debido a que HTN no hace uso de funciones de heurística que estimen el costo restante para llegar al objetivo, sino que trabaja desde refinamientos de acciones de alto nivel.

Análisis del Punto 5b – Misiones múltiples

Complejidad espacial $O(b^d)$: La complejidad espacial depende del factor de ramificación, representado por b , y de la profundidad del plan, representada por d . El algoritmo debe almacenar planes parciales, considerando varios pacientes, suministros y los posibles órdenes de rescate.

Complejidad temporal $O(b^d)$: La complejidad temporal depende del factor de ramificación, representado por b , que corresponde al número de acciones posibles desde un estado, y de la profundidad del plan, representada por d . El algoritmo debe almacenar planes parciales, creciendo la cantidad de estados y refinamientos según el número de pacientes.

Complejidad: El algoritmo sí se puede considerar completo, puesto que puede encontrar una solución en caso de que esta exista, siempre que los refinamientos definidos permitan llegar a acciones primitivas desde HLAs, para así rescatar a todos los pacientes.

Optimalidad: El algoritmo no garantiza encontrar el plan de menor costo, no pudiendo considerarse óptimo, debido a que el resultado depende del orden en el que se procesan los pacientes, y de los refinamientos definidos. Puede encontrar un plan válido, pero este no necesariamente es el más óptimo posible.

¿La heurística es admisible? La heurística no se puede considerar admisible, ya que en HTN no se usa una heurística de estimación como A^* en el 4b. El algoritmo se guía por la derivación de HLAs y refinamientos.

¿La heurística es consistente? La heurística no se puede considerar consistente, porque no tiene una función heurística que calcule el costo restante. La planificación depende de la estructura jerárquica de las tareas.

Impacto del orden de los pacientes: En `MultiRescueProblem`, el orden en que se procesan los pacientes afecta linealmente la longitud del plan: cada misión añade aproximadamente los pasos de una misión individual más los desplazamientos necesarios entre los puntos de pickup y el puesto médico. La Tabla 2 muestra empíricamente que dos layouts distintos con 2 pacientes (`tinyMulti` y `duoRescue`) producen planes de longitudes diferentes (21 y 20 acciones respectivamente). Un mal orden (zigzag entre pacientes alejados) puede aumentar el plan en un 30–50 % comparado con un orden “vecino-más-cercano”. En esencia, optimizar el orden de los pacientes equivale a resolver un *Problema del Vendedor Viajero (TSP)* sobre las ubicaciones de los pacientes, lo cual HTN no resuelve óptimamente sin una heurística adicional. Nuestra implementación procesa los pacientes en el orden en que aparecen en el layout y solo realiza `PrepareSupplies` en la primera misión (las siguientes reutilizan el fuente `SuppliesReady(m)` ya activado).

Ventaja del planificador HTN sobre el planificador clásico

El planificador HTN reduce la complejidad del problema, al dividir las tareas grandes en subtarefas más pequeñas y específicas con refinamientos jerárquicos. Esto lo que hace es que evita explorar todas las acciones posibles desde cada estado, como sí ocurre en el planificador clásico; en HTN se limita la búsqueda a las acciones más relevantes para solucionar el objetivo propuesto, reduciendo los espacios de búsqueda y mejorando la eficiencia en problemas complejos. Empíricamente, HTN resuelve `smallMulti` (3 pacientes) en menos de 2 ms, mientras que un planificador clásico equivalente sería computacionalmente intratable para múltiples pacientes.

Comparación empírica

A continuación se presentan los resultados de ejecutar cada planificador sobre los layouts de prueba. Las métricas reportadas son: longitud del plan ($|\pi|$), número de estados expandidos (Exp) y tiempo de ejecución en segundos (t).

Análisis cualitativo de la Tabla 1:

- **Todos los algoritmos encuentran el mismo plan óptimo** ($|\pi|$ coincide en cada layout), lo cual confirma empíricamente la optimalidad de BFS y A^* con heurísticas admisibles.

Cuadro 1: Comparación de planificadores clásicos en `layouts/simple/` (`SimpleRescueProblem`).

Layout	Algoritmo	$ \pi $	Exp	t (s)
tinyBase	forwardBFS	9	225	0.016
	backwardSearch	9	0*	0.005
	A* + ignorePrec	9	138	0.589
	A* + ignoreDel	9	138	0.559
cornerRescue	forwardBFS	19	1470	0.485
	backwardSearch	19	0*	1.940
	A* + ignorePrec	19	925	28.356
	A* + ignoreDel	19	925	42.999
smallRescue	forwardBFS	23	1181	0.845
	backwardSearch	23	0*	23.806
	A* + ignorePrec	23	960	43.143
	A* + ignoreDel	23	960	64.373
mediumRescue	forwardBFS	32	5148	4.630

* Backward search no expande estados *del problema* sino subobjetivos; su contador `_expanded` permanece en 0 porque opera sobre la regresión, no sobre `getSuccessors`. El tiempo refleja el costo real.

Cuadro 2: Planificador HTN en `layouts/htn/` y `layouts/multi/`.

Problema	Layout	# pacientes	$ \pi $	Exp [†]	t (s)
Simple + HTN	tinyHTN	1	9	—	0.001
	htnBase	1	12	—	0.001
Multi + HTN	tinyMulti	2	21	—	0.001
	duoRescue	2	20	—	0.001
	smallMulti	3	41	—	0.001

† El contador `_expanded` del problema se mantiene en 0 para HTN porque `hierarchicalSearch` no expande estados clásicos vía `getSuccessors`: opera sobre el espacio de *refinamientos jerárquicos*. La eficiencia se evidencia en el tiempo de ejecución (3–4 órdenes de magnitud por debajo del planificador clásico en layouts comparables).

- **forwardBFS es la opción más rápida en la práctica** para este dominio: su simplicidad gana frente al costo por nodo de las heurísticas.
- **A* explora menos nodos** que BFS ($\approx 38\%$ menos en `cornerRescue` y $\approx 19\%$ menos en `smallRescue`), confirmando que las heurísticas *sí* son informativas. Sin embargo, el costo por evaluación de la heurística (cada llamada a `ignorePreconditionsHeuristic` o `ignoreDeleteListsHeuristic` enumera todos los groundings) penaliza el tiempo total.
- **Las dos heurísticas son equivalentes en estos layouts** (mismo número de expansiones). Esto se explica porque el goal es un único fluente que ninguna acción “deshace”: la relajación de listas de borrado no aporta información adicional frente a ignorar precondiciones para este goal específico.
- **backwardSearch es competitivo en tinyBase** (el objetivo es muy específico) pero degrada rápidamente: en `smallRescue` es $28\times$ más lento que BFS porque el espacio de subobjetivos relevantes crece.

Análisis cualitativo de la Tabla 2:

- **HTN encuentra planes óptimos para los layouts simples:** en `cornerRescue` HTN produce 19 acciones, igualando el plan óptimo de `forwardBFS`. Esto valida que la jerarquía propuesta (`Navigate`, `PrepareSupplies`, `ExtractPatient`, `FullRescueMission`) captura correctamente la estructura del dominio.

- **Escalabilidad superior en multi-rescate:** mientras los planificadores clásicos se vuelven prohibitivos al crecer el número de pacientes (por la explosión combinatoria del espacio de estados), HTN resuelve `smallMulti` con 3 pacientes en 41 acciones y menos de 2 ms. La longitud del plan crece aproximadamente como $|\pi| \approx k \cdot N_{\text{pacientes}}$, donde $k \approx 10\text{--}14$ depende del tamaño del layout.
- **El orden de pacientes importa:** `tinyMulti` produce un plan de 21 pasos para 2 pacientes y `duoRescue` (también con 2 pacientes) produce 20, evidenciando que la geometría y el orden en que se procesan los pacientes afectan directamente la longitud del plan. Nuestra implementación procesa los pacientes en el orden en que aparecen en el layout, lo cual no es necesariamente óptimo (equivaldría a resolver un TSP).
- **Tiempos casi constantes:** HTN ejecuta en menos de 2 ms en todos los layouts probados porque la jerarquía limita drásticamente el espacio de búsqueda — los refinamientos están parametrizados por las ubicaciones concretas del layout y la búsqueda BFS sobre refinamientos converge inmediatamente.

Punto 6b: Reflexión sobre el uso de IA generativa

La IA, como se ha estudiado extensamente en el curso, ha incursionado en muchísimos ámbitos tanto de la vida cotidiana como en ambientes profesionales o académicos. Todo apunta a que su prominencia no va a hacer más que incrementar en el futuro próximo.

Sin embargo, como cualquier herramienta, hay que tener consideración en los momentos y lugares en los que se usa. Durante la elaboración del proyecto, las implementaciones de las funciones base de los puntos 1 a 5 (`is_applicable`, `apply_action`, `get_applicable_actions`, `forwardBFS`, `regress`, `aStarPlanner`, las heurísticas `ignorePreconditionsHeuristic` e `ignoreDeleteListsHeuristic`, y el planificador HTN) fueron realizadas en su totalidad por los miembros del grupo a partir del esqueleto provisto.

El uso de agentes de IA se limitó a tres aspectos puntuales: (1) la depuración de `backwardSearch` (la versión inicial no terminaba; el prompt utilizado y la versión inicial están registrados como comentarios al final de `planning/planner.py`), (2) sugerencias menores de legibilidad sobre el código ya escrito y (3) la redacción y formateo del presente informe en $\text{\LaTeX}$.

Es extremadamente importante que en espacios de desarrollo de software se tenga una base robusta de entendimiento conceptual antes de emplear la ayuda de LLMs: cuando se nos sugería un cambio, podíamos juzgar si era razonable y reproducirlo si fuera necesario en una sustentación. La principal lección obtenida es que la IA acelera tareas mecánicas (formateo, búsqueda de bugs evidentes, redacción), pero el diseño algorítmico (decidir, por ejemplo, qué flujos estáticos podar en `backwardSearch` o cómo estructurar la jerarquía HTN) sigue siendo un trabajo de razonamiento humano que la IA puede acompañar pero no reemplazar.